UNIVERSITY OF ST. LA SALLE

IT Helpdesk

INFORMATION ASSURANCE AND SECURITY · LEARNING EVIDENCE

Secure Login System

Authentication, Authorization and Password Hashing
in the USLS IT Support Helpdesk

GROUP MEMBERS

Parcon, Rayle

Paza, Ezra Joie

Punzalan, Khirk Justin

Plateros, Glory Mae

AT A GLANCE

PHP 8 (layered, object-oriented) and MySQL / MariaDB

Argon2id hashing, two-factor sign-in, three-role RBAC

149 of 149 automated security checks passing

Updated October 20, 2026

RELIGIO · MORES · CULTURA

Contents

1. System Overview
2. Technical Architecture and Stack
3. Core Security Implementation
   A. Password Hashing
   B. Authentication
   C. Authorization
4. Additional Web Security Controls
5. Application Workflow and Page Directory
6. Testing and Verification
7. What We Changed and Why
8. Suggestions for Further Improvement
9. Installation Guide

1. System Overview

The USLS IT Support Helpdesk is an institutional web application for the University of St. La Salle. Students, faculty and staff use it to report campus technology problems (Wi-Fi and network issues, LMS and portal access, computer laboratory hardware, software licensing and institutional accounts) and to follow each request until it is resolved. IT technicians work through a shared ticket queue, and administrators manage accounts and review a security audit trail.

Because the system holds personal accounts and controls who can see and change support records, the project focuses on a **secure login system** built on three pillars: **password hashing**, **authentication** and **authorization**. Every request passes through these controls, and an automated test suite attacks the running system to prove that each one works.

> **Three user roles.** *Student / Staff* file and track their own tickets. *IT Technicians* manage the ticket queue. *Administrators* also manage user accounts and the audit log. New accounts always start as Student / Staff; only an administrator can grant more access.

2. Technical Architecture and Stack

Layer	Technology
Backend	PHP 8 with strict types, namespaced classes and a PSR-4 autoloader (no external libraries)
Database	MySQL / MariaDB through PDO with native prepared statements; foreign keys enforce relationships
Frontend	HTML5, CSS3 and a small amount of plain JavaScript; La Salle green and gold design
Web server	Apache (XAMPP); only the `public/` folder is reachable from the browser
Testing	Custom end-to-end and unit test runner (`tests/run.php`) using cURL against a live test server

Layered design

The first version mixed SQL, security checks and HTML in every page. We reorganised the code into layers so that each security rule lives in exactly one place and can be tested on its own:

```
Browser
   │  HTTPS request
   ▼
public/              Thin page controllers. The ONLY folder the web server exposes.
   │                 Read input → call a service → render a view.
   │
   ├── src/Auth/          Sign-in flow, two-factor, registration, RBAC, admin actions
   ├── src/Security/      Hashing, password policy, CSRF, rate limiting, AES encryption,
   │                      TOTP, audit log, HTTP security headers, secret keys
   ├── src/Domain/        Ticket rules (allowed values and validation)
   ├── src/Repositories/  Every SQL query, always with bound parameters
   └── src/Core/          Configuration, database connection, hardened session, views
   ▼
views/               HTML templates only. Every value is escaped before output.
config/              Settings, plus secrets.php (random keys, never committed to Git)
```

- **Defence in depth:** source code, configuration and secret keys sit outside the web root. Even if the web server were misconfigured, each private folder also has its own "deny all" rule.
- **Single source of truth:** all permissions are defined in `Rbac::PERMISSIONS` ; all SQL is in the repositories; all output passes through one escaping function.
- **Testable:** each security class (for example `PasswordHasher` or `Totp`) is checked directly by the test suite.

3. Core Security Implementation

Security is built into the lifecycle of every request through three pillars.

A. Password Hashing (cryptographic security)

Passwords are never stored or logged in readable form. Each password is first combined with a secret *pepper* using HMAC-SHA256, then hashed with **Argon2id**, the winner of the Password Hashing Competition. Argon2id is deliberately slow and memory-hard (64 MB per hash), which makes guessing attacks on a stolen database very expensive. A unique random salt is generated for every password automatically.

```
// src/Security/PasswordHasher.php
public static function hash(string $password): string
{
    [$algo, $options] = self::algorithm();   // Argon2id, 64 MB, 4 iterations
    return password_hash(self::pepper($password), $algo, $options);
}

public static function pepper(string $password): string
{
    // the key is kept outside the database
    return hash_hmac('sha256', $password, Secrets::get('pepper'));
}
```

- **Pepper:** the secret key lives in `config/secrets.php`, not in the database. An attacker who steals only the database cannot even begin to guess passwords.
- **Automatic upgrade:** hashes created by the first version (plain bcrypt) are still accepted once, then re-hashed with Argon2id at the next sign-in.
- **Timing-safe:** when a username does not exist, a dummy hash is still checked, so attackers cannot tell real accounts apart by response time.
- **Password policy (server-side):** 12 to 128 characters with upper and lower case, a number and a symbol; common passwords, long repeated characters and passwords containing the username or e-mail name are rejected. The live strength meter in the browser is only a guide.

```
// Verification never decrypts anything; it re-hashes the attempt and compares.
[$valid, $needsRehash] = PasswordHasher::verify($password, $account['password']);
```

B. Authentication (verifying who you are)

The sign-in flow is handled by `Auth::attempt()` and protects the session for its whole lifetime.

Control	How it works
Brute-force lockout	5 failed attempts per account, or 50 from one network address (generous, because a whole campus can share one public IP), within 15 minutes blocks sign-in. The lockout behaves identically for usernames that do not exist, and a wrong password takes exactly as long to reject as an unknown username, so neither messages nor timing reveal which accounts exist.
Generic error messages	A wrong password and an unknown username give the same message: "Incorrect username or password."
Two-factor authentication	Optional 6-digit codes from Google or Microsoft Authenticator, implemented from the RFC 6238 standard and verified against its official test vectors. The shared secret is stored encrypted with AES-256-GCM. A code is accepted only once, even if two requests arrive at the same moment, and 5 wrong codes lock the account. The lockout is checked before every code, so opening many sign-in sessions gives no extra guesses. Turning 2FA on requires the password, and an existing setup cannot be silently replaced.
Recovery codes	8 single-use backup codes, stored hashed like passwords and shown to the user only once.
Session fixation defence	A brand-new session ID is issued at sign-in and every 10 minutes; session IDs the server did not create are refused (strict mode).
Hardened cookie	`HttpOnly` (JavaScript cannot read it), `SameSite=Strict` (not sent from other websites), `Secure` on HTTPS, never placed in URLs.
Session binding	The session is tied to the browser that created it. A stolen cookie replayed from another browser is rejected and logged as a hijack attempt.
Timeouts	15 minutes of inactivity or 8 hours in total ends the session.
Global sign-out	Changing a password, or an administrator changing a role or deactivating an account, instantly ends every other session of that user.
Step-up authentication	User management and the audit log ask for the password again if it was not confirmed in the last 10 minutes.
Rate-limited password checks	Every screen that asks for the current password (step-up, password change, two-factor changes) counts wrong guesses; after 5 the session is ended, so an unattended computer cannot be used to guess the password.
Safe logout	Sign-out only accepts a POST request with a CSRF token, so another website cannot force a logout.

```
// src/Auth/Auth.php (excerpt): new session ID at sign-in prevents session fixation
public static function login(array $user): void
{
    Session::renew([                       // session_regenerate_id(true) + fresh data
        'user_id'         => (int) $user['id'],
        'role'            => $user['role'],
        'session_version' => (int) $user['session_version'],
        'fingerprint'     => Session::fingerprint(),
        'login_time'      => time(),
        // ...
    ]);
}
```

C. Authorization (controlling what you can access)

Access is controlled with **role-based access control (RBAC)**. Pages never check a role name directly; they ask for a *permission*, and the mapping from roles to permissions is defined once in `src/Auth/Rbac.php`.

Permission	Student / Staff	IT Technician	Administrator
File and view own tickets	✓		
Manage the ticket queue		✓	✓
Support desk overview		✓	✓
Manage users and roles			✓
View and verify the audit log			✓

```
// Every protected page starts with one line, before any data is read or changed:
$user = Auth::requirePermission('ticket.manage');

// Inside: not signed in → redirect to sign-in; signed in without the permission →
// HTTP 403 "Access denied", and the attempt is written to the audit log.
```

- **Always current:** the user's role and status are re-read from the database on every request, so a demotion or deactivation takes effect on the very next click.
- **Insecure direct object references (IDOR):** ownership is part of the SQL itself, e.g. `DELETE FROM tickets WHERE id = ? AND user_id = ? AND status = 'Open'`. Changing a ticket number in a request cannot reach someone else's ticket.
- **No privilege escalation at sign-up:** new accounts are always Student / Staff; a forged `role=admin` field is ignored.
- **Safety rules for administrators:** an admin cannot demote, deactivate or delete their own account, and the last active administrator cannot be removed.
- **Dynamic interface:** the sidebar only shows the pages the user's permissions allow.

4. Additional Web Security Controls

Threat	Protection
SQL injection	Every query uses PDO prepared statements with emulation turned off, so input is never part of the SQL text. Filter values (status, priority, role) are checked against fixed whitelists.
Cross-site scripting (XSS)	All output goes through `e()` (`htmlspecialchars`). A strict Content-Security-Policy additionally blocks inline and third-party scripts, so even a missed escape could not run code.
Cross-site request forgery (CSRF)	Every form carries a random 256-bit token tied to the session, compared in constant time. Requests without it are refused with HTTP 400 and logged.
Clickjacking	`X-Frame-Options: DENY` and `frame-ancestors 'none'`.
Other headers	`X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`, `Cache-Control: no-store`, HSTS when HTTPS is used, and no `X-Powered-By`.
Open redirects	After password confirmation, only a fixed list of internal pages is accepted as a destination.
Password spraying and mass sign-ups	Limits per network address for sign-in failures (50 per 15 minutes) and new accounts (5 per hour).
Hostile input	Very long values and invalid UTF-8 bytes are cleaned before they reach the database, so they cannot cause errors or corrupt the audit trail.
Information leakage	PHP errors are never displayed; database errors are logged and the visitor sees a generic message.
Exposed source and secrets	Only `public/` is web-reachable; `src/`, `config/`, `views/`, `database/`, `setup/` and `tests/` return 403 or 404.

Tamper-evident audit log

Every security event (sign-ins, failures, lockouts, denied pages, role changes, password changes, 2FA changes) is recorded with the user, IP address and time. Each row also stores an HMAC of its contents chained to the previous row, like a simple blockchain. The administrator's *Verify integrity* button recalculates the chain; if anyone edits or deletes a row directly in the database, the exact entry is reported. The HMAC key is not stored in the database, so the chain cannot be forged from there.

```
$payload = implode('|', [$prevHash, $userId, $username, $event, $details, $ip, $userAgent,
$createdAt]);
$rowHash = hash_hmac('sha256', $payload, Secrets::get('encryption_key'));
```

5. Application Workflow and Page Directory

Page	Who	Purpose
index.php	Everyone	Landing page with the La Salle branding, help categories and security summary.
register.php	Visitors	Account creation with server-side password policy and a live strength meter.
login.php	Visitors	Sign-in by username or e-mail, with lockout and generic errors.
verify_2fa.php	Users with 2FA	Second sign-in step: authenticator code or one-time recovery code.
dashboard.php	Student / Staff	Summary of requests and the form for filing a new ticket.
my_tickets.php	Student / Staff	History of the user's own tickets; open tickets can be withdrawn.
account.php	All signed-in users	Change password, set up two-factor, recovery codes, sign out other devices, recent activity.
admin_dashboard.php	Technician, Admin	Queue overview; administrators also see security statistics and recent alerts.
admin_tickets.php	Technician, Admin	Search, filter, claim and update tickets, sorted by status and priority.
manage_users.php	Admin (password re-confirmed)	Change roles, deactivate, clear lockouts, reset 2FA, sign out or delete accounts.
audit_log.php	Admin (password re-confirmed)	Search the security trail and verify its integrity.
confirm_password.php	Signed-in users	Step-up authentication before sensitive pages.
logout.php	Signed-in users	POST-only sign-out that destroys the session.

Sign-in sequence

```
1. User submits username + password (with CSRF token)
2. Lockout check (account and network address)      → too many failures: "Too many attempts"
3. Argon2id verification (dummy check if no account) → wrong: generic error, failure recorded
4. Account active?                                    → no: "account deactivated"
5. 2FA enabled?  yes → verify_2fa.php: code checked, replay blocked, 5 failures lock the account
6. New session ID issued, session bound to browser, event written to the audit log
7. Redirect to the dashboard for the user's role
```

6. Testing and Verification

To check every feature and security control, we wrote an automated test suite (`tests/run.php`). It builds a separate test database from the schema, starts its own web server, and then behaves like both a normal user and an attacker: it submits forms, forges requests, replays cookies and codes, injects SQL and scripts, and inspects the database afterwards. In addition, every page was driven in a real web browser (64 checks covering forms, menus, filters, confirmation dialogs, two-factor setup with real codes, links for every role, and phone layouts), and the code was reviewed line by line for security and front-end defects. Both test suites were then run three times in a row with identical results.

149 / 149 automated checks passed on October 20, 2026.
Command: `php tests/run.php`

Area	Passed	Examples of what is checked
Password hashing	7 / 7	Passwords are hashed with Argon2id; Correct password verifies; Wrong password is rejected
Password policy	7 / 7	Rejects short passwords; Rejects passwords without an uppercase letter; Rejects passwords without a symbol
Two-factor (TOTP, RFC 6238)	6 / 6	RFC 6238 test vector at T=59 gives 287082; RFC 6238 test vector at T=1111111109 gives 081804; A current code is accepted
Encryption (AES-256-GCM)	3 / 3	Encrypt then decrypt returns the original; Ciphertext does not contain the secret; Tampered ciphertext is detected and refused
Authorization rules and input handling	7 / 7	Students can file tickets but not manage them; Technicians manage tickets but not users; Administrators can manage users and read the audit log
HTTP security headers and file protection	13 / 13	Content-Security-Policy blocks inline and third-party scripts; X-Frame-Options: DENY (clickjacking); X-Content-Type-Options: nosniff
Registration	7 / 7	Weak password is rejected on the server; Mismatched confirmation is rejected; Invalid username and e-mail are rejected
CSRF protection	8 / 8	Sign-in without a CSRF token is refused (400); Sign-in with a forged CSRF token is refused (400); Registration without a CSRF token is refused (400)
Authentication	15 / 15	Wrong password shows a generic message; Unknown username shows the identical message (no account enumeration); Sign-in with e-mail address works
Brute-force protection	4 / 4	After 5 failures even the correct password is refused; Lockout is written to the audit log; Non-existent accounts lock out the same way (no enumeration)

Role-based access control (every page, every role)	10 / 10	dashboard.php student 200, technician 302, admin 302; my_tickets.php student 200, technician 403, admin 403; account.php student 200, technician 200, admin 200
Tickets: validation, XSS, SQL injection and ownership	12 / 12	A valid ticket is created; Invalid category, priority and lengths are rejected; Stored XSS payload is escaped on My tickets
Account administration	9 / 9	Role change is saved; The user is signed out immediately after a role change; Unknown roles are refused
Step-up authentication and redirects	4 / 4	User management asks for the password again after the step-up window; A wrong confirmation password is refused; The correct password unlocks the page
Password change	8 / 8	Wrong current password is refused; A weak new password is refused; Password is changed
Two-factor authentication	21 / 21	Starting 2FA setup requires the correct password; A 160-bit setup key is shown; A wrong setup code does not enable 2FA
Audit log integrity	3 / 3	The untouched log verifies as intact; Security events are being recorded (122 so far); Editing a log row directly in the database is detected
Session timeout	2 / 2	Signed in; An idle session expires automatically
Rate limits per network address	2 / 2	Mass account creation from one address is blocked; Password spraying across many accounts locks the address
Information leakage	1 / 1	No PHP errors, warnings or stack traces in any response

Defects found during verification, and fixed

Problem found	Fix
An attacker who knew a password could open many half-finished sign-ins and keep guessing two-factor codes after the account locked	The lockout is now checked before any code is tried
Unknown usernames took about twice as long to reject as wrong passwords, revealing which accounts exist	A fixed hash with identical cost settings is checked for unknown users; old-format hashes are only tried for old accounts
Two-factor lockout missed failures in the same second as a correct password	Failures are counted by record order instead of one-second timestamps
Password checks on the account page had no guess limit	All of them go through the same rate-limited check as step-up authentication

Two-factor could be enabled or replaced without the password	Setup requires the password and is refused while 2FA is already on
A half-finished sign-in survived a password change	Pending sign-ins remember the session version and the browser, and are cancelled when either changes
Two simultaneous requests could reuse one code or recovery code	Codes are claimed with a single atomic database update
Very long or invalid UTF-8 usernames could cause errors or a false "tampered" audit result	Input is cleaned and cut to the column size before storage
One student's failed sign-ins could lock out everyone sharing the campus IP	The per-address limit was raised to a level that still stops password spraying
The setup script could promote the wrong account if an e-mail was already in use	It looks accounts up by username only and refuses e-mail conflicts
Front end: ticket search did nothing, the user filter matched everyone, the strength meter disagreed with the server rules, footer links led staff to an error page, pasting a spaced code failed, the hidden mobile menu could be reached by keyboard	All corrected and covered by the browser test

7. What We Changed and Why

Area	First version	Current version
Password hashing	`PASSWORD_DEFAULT` (bcrypt, cost 10)	Argon2id (64 MB) plus a secret pepper; old hashes upgraded automatically
Password rules	Any non-empty password	Server-side policy with common-password list and strength meter
Brute-force protection	None	Lockout per account and per network address, identical for unknown users
Two-factor sign-in	None	TOTP authenticator codes, encrypted secrets, replay protection, recovery codes
Sessions	ID regenerated at sign-in	Strict mode, hardened cookie, idle and absolute timeouts, browser binding, global sign-out
Roles	user, admin (checked page by page)	Student, technician, admin with central permission map, re-checked every request
CSRF	None; deletes used plain GET links	Token on every form; all state changes are POST only
Insecure scripts	`debug_login.php`, `setup_admin.php`, `create_admin.php` reset admin to `admin123` from any browser	Removed; administrators are created with a command-line script that enforces the password policy
XSS	Some values (status, messages) printed unescaped	Every value escaped, plus Content-Security-Policy
Database credentials	Hard-coded root account	Configurable through environment variables; least-privilege account provided
Error handling	Database error messages shown to visitors	Errors logged privately, generic message shown
Audit trail	None	Tamper-evident HMAC-chained log with integrity check
Code structure	SQL, logic and HTML mixed in each page	Layered architecture; only `public/` is web-reachable
Testing	Manual only	149 automated checks covering every control
Interface	Glass-style cards, inline styles	Clean, spacious La Salle design with sidebar, branded logo and mobile support

8. Suggestions for Further Improvement

1. **HTTPS everywhere.** Enable SSL and the redirect already prepared in `public/.htaccess`; cookies then receive the `Secure` flag and HSTS is sent automatically.
2. **E-mail verification and password reset** using one-time, expiring, hashed tokens.
3. **Breached-password check** with the Have I Been Pwned range API, which only sends the first five characters of a hash.
4. **Passkeys (WebAuthn)** for phishing-resistant sign-in with fingerprint, Windows Hello or a security key.
5. **QR code for two-factor setup**, generated locally, so users can scan instead of typing the key.
6. **Single sign-on** with the university's Microsoft 365 accounts through OAuth 2.0 / OpenID Connect.
7. **Least-privilege database account** (`helpdesk_app` in the schema) that can only add to the audit log, never change it.
8. **CAPTCHA after repeated failures** to slow automated attacks without locking out real users.
9. **Off-site log storage** so that even a full server compromise cannot erase the security trail.
10. **Continuous integration:** run `tests/run.php` automatically on every push.

9. Installation Guide

1. Copy the project folder into `C:\xampp\htdocs\` and start Apache and MySQL.
2. In phpMyAdmin, import `database/schema.sql` (it also upgrades a database from the first version).
3. Create the first administrator: `C:\xampp\php\php.exe setup\create_admin.php`
4. Open `http://localhost/<folder>/` in the browser and sign in.
5. Optional: run `C:\xampp\php\php.exe tests\run.php` to verify every security control.

The file `config/secrets.php` is generated on first use and must never be shared or committed to version control.